

Software Design Document

When designing a notification system in a system design interview, it's important to systematically address the problem by considering requirements, constraints, architecture, and detailed design aspects. Here's a structured approach:

Step 1: Gather Requirements

Functional Requirements

- **Multi-Channel Support**: Deliver notifications through Email, SMS, and Push notifications.
- **Real-time and Scheduled Delivery**: Support immediate and scheduled notification dispatch.
- **User Preferences**: Allow users to customize notification settings and preferences.
- **High Volume Handling**: Capable of handling millions of notifications.
- **Failure Management**: Retry on delivery failure and log unsuccessful attempts.

Non-Functional Requirements

- **Scalability**: Seamlessly scale with increased load.
- **Reliability**: Ensure high availability and fault tolerance.
- **Low Latency**: Quick delivery for real-time notifications.
- **Extensibility**: Easily add new notification channels or features.
- **Security**: Protect user data and ensure secure communication.

Step 2: Conceptualize High-Level Architecture

Key Components

1. **Event Source**: Application or system triggers generating notification events.
2. **Message Queue**: Buffer notifications to ensure eventual processing and reliability.
3. **Notification Service**: Central hub that processes requests and manages business logic.

4. **Channel Provider**: Handles delivery specifics for Email, SMS, and Push.
5. **Preference Management**: Stores and processes user notification preferences.
6. **Template Engine**: Manages notification templates for consistent formatting.
7. **Scheduler**: Handles scheduled notifications.
8. **Database**: Persists user settings, templates, logs.

Step 3: Architecture Details

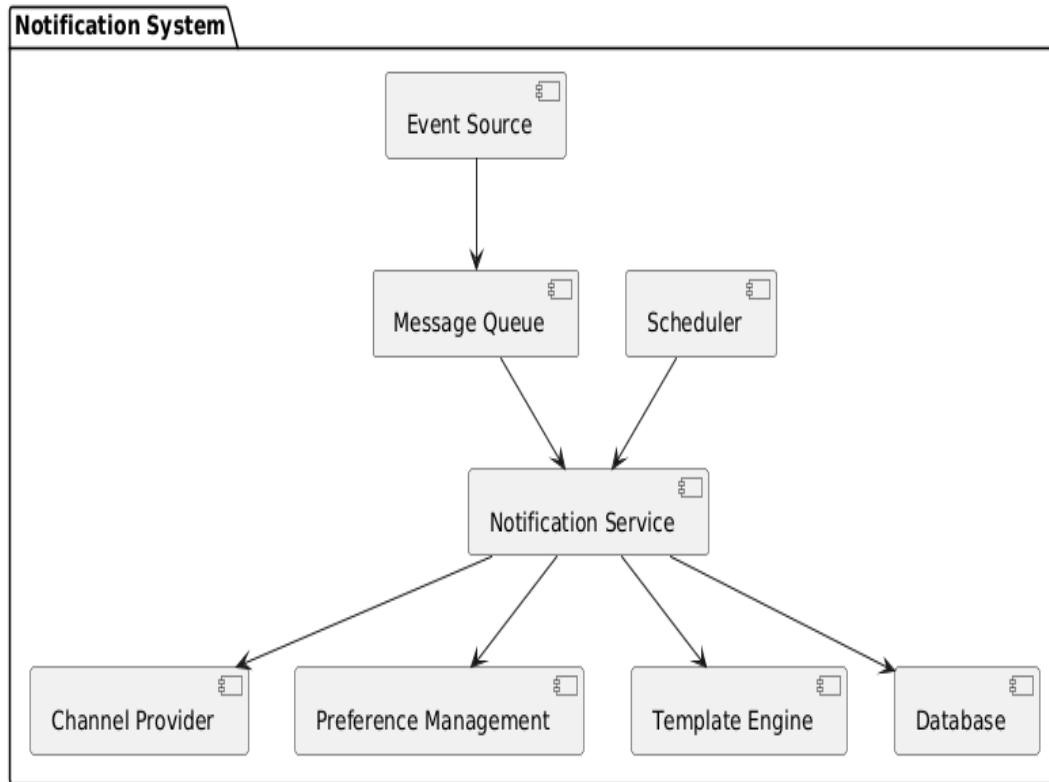
Data Flow

1. **Event Generation**: Notifications originate from application events, sent to a queue.
2. **Processing Queue**: Messages are queued to handle sudden spikes in traffic.
3. **Notification Processor**: Consumes messages, checks user preferences, and formats content using templates.
4. **Channel Distribution**: Sends notifications to the appropriate channel providers.
5. **Retry Logic**: Manages retries and fallbacks for failed deliveries.

Step 4: High-Level Architecture Diagram

Here's a PlantUML diagram illustrating the proposed architecture:

```
``plantuml
```



...

Step 5: Considerations and Trade-offs

- **Scalability**: Use cloud-based solutions (e.g., AWS SNS, SQS) for scale and resilience.
- **Data Consistency**: Ensure preferences are reliably accessed and updated.
- **Monitoring**: Implement logging and monitoring tools to track system performance and errors.
- **Extensibility**: Ensure modularity so new channels can be integrated with minimal disruption.

Conclusion

The design encompasses key concepts of reliability, extensibility, and performance necessary for a scalable notification system. This foundation allows for future expansion and adaptation to evolving requirements and technologies.